

Initial MVC Diagram:

[Browser / Client]

|

v

[Express Router: /manager]

|

v

[Manager Controller]

- Loads schedule + featured DJ
- Validates schedule form data
- Retrieves reports + stats
- Sends updates back to DB

|

v

[Mongoose Models]

- Manager Profile (User ID, schedule, song reports)
- Genre (Name, description, selected/not selected)

|

v

[MongoDB Database]

Description:

The client sends HTTP GET/POST requests to /manager.

The Manager Controller queries multiple models (Manager Profile, Genre), gets dashboard stats, and loads the Manager dashboard as an EJS template.

POST requests update the schedule via Mongoose and return JSON for UI updates.

Date Created: 11/27/25

Initial Sequence Diagram:

Client -> Router: GET /manager

Router -> Manager Controller: fetch dashboard data by sessionID

Manager Controller -> MongoDB: find or insert new manager profile

MongoDB -->getManagerProfile: return documents

Router -> Genre: find and sort data

Genre -> Router: create genre list

Router -> EJS view: render ejs file

Ejs view -> client: frontend(HTML, CSS, JS)

Client -> Express Router: POST /manager/add-schedule (JSON payload)

Router -> Body Parser: parse JSON

Router -> getManagerProfile: attach manager profile to req

Router: validate fields (date, time, DJ name, genre)

Router: check duplicates + past date

Router -> ManagerProfile Model: addToSchedule() + save()

ManagerProfile Model -> MongoDB: update document

MongoDB -> Router: confirmation

Router -> Client: JSON {schedule, stats}

Client -> Express Router: POST /manager/remove-schedule

Router -> Body Parser

Router -> getManagerProfile

Router: Validate index

Router -> ManagerProfile Model: removeFromSchedule() + save()

ManagerProfile Model -> MongoDB: update document

MongoDB -> Router

Router -> Client: JSON {schedule, stats}

The manager dashboard loads by retrieving the sessionID in managerprofile, or using a default if no values are found.

The server finds DJs that are featured/upcoming, calculates the dashboard stats, loads genres, and renders the frontend using EJS.

When the manager changes the schedule, the frontend sends a request to the server. Then the server applies the change and sends back the updated schedule and updated statistics so that the UI refreshes with the updated information.

Diagrams created: 11/27/25

Updated Diagrams:

Component Diagram:

[Browser / Client]

- HTTP GET/POST requests

|

v

[Express Router: /manager]

- getManagerProfile() runs, finds or creates profile with sessionID
- Attaches to req.managerProfile

|

v

[Manager Controller]

- Middleware: getManagerProfile (loads/creates profile by sessionID)
- GET /: Loads dashboard, fetches genres, calculates stats
- POST /add-schedule: Validates and adds DJ to schedule
- POST /remove-schedule: Removes DJ from schedule
- GET /schedule-data: Returns current data

|

v

[Mongoose Models]

- Manager Profile (User ID, schedule, song reports)
- Genre (Name, description, selected/not selected)

|

v

[MongoDB Database]

Sequence Diagram

Client -> Router: POST /manager/add-schedule {date, time, djName, genre}

Router -> getManagerProfile: attach profile

Router: validate all fields required

Router: validate name pattern (letters/spaces only)

Router: validate date not in past

Router: check for duplicate DJ names

Router: calculate timeSlot

Router -> ManagerProfile: addToSchedule()

ManagerProfile -> MongoDB: save updated schedule

MongoDB -> Router: success

Router -> Client: JSON {success, schedule, stats}

Client: update table and stats on page

Client -> Router: POST /manager/remove-schedule {index}

Router -> getManagerProfile: attach profile

Router: validate index
Router -> ManagerProfile: removeFromSchedule(index)
ManagerProfile -> MongoDB: save updated schedule
MongoDB -> Router: success
Router -> Client: JSON {success, schedule, stats}
Client: update table and stats on page

The updated diagrams show how the middleware function runs first to grab the user's profile from the database using their session ID. I added all the routes to the MVC diagram and included the actual fields from my Mongoose models. The sequence diagrams now show how the server calculates which DJ is featured and upcoming, plus it uses demo data from a default profile if the user hasn't added any DJs yet, and everything updates on the page without having to refresh.

Date Created: 12/01/25